3.2.3.1 The Set-Intersection Operation

The first additional-relational algebra operation that we shall define is **set intersection** ($\cap$). Suppose that we wish to find all customers who have both a loan and an account. Using set intersection, we can write

$$\Pi_{customer\text{-}name}\ (borrower)\ \cap\ \Pi_{customer\text{-}name}\ (depositor)$$

The result relation for this query appears in Figure 3.20.

Note that we can rewrite any relational algebra expression that uses set intersection by replacing the intersection operation with a pair of set-difference operations as:

$$r\ \cap\ s\ =\ r\ -\ (r\ -\ s)$$

Thus, set intersection is not a fundamental operation and does not add any power to the relational algebra. It is simply more convenient to write $r\ \cap\ s$ than to write $r\ -\ (r\ -\ s)$.

3.2.3.2 The Natural-Join Operation

It is often desirable to simplify certain queries that require a Cartesian product. Usually, a query that involves a Cartesian product includes a selection operation on the result of the Cartesian product. Consider the query "Find the names of all customers who have a loan at the bank, along with the loan number and the loan amount." We first form the Cartesian product of the *borrower* and *loan* relations. Then, we select those tuples that pertain to only the same *loan-number*, followed by the projection of the resulting *customer-name*, *loan-number*, and *amount*:

$$\Pi_{customer\text{-}name,\ loan.loan\text{-}number,\ amount}\ (\sigma_{borrower.loan\text{-}number\ =\ loan.loan\text{-}number}\ (borrower\ \times\ loan))$$

The *natural join* is a binary operation that allows us to combine certain selections and a Cartesian product into one operation. It is denoted by the "join" symbol $\bowtie$. The natural-join operation forms a Cartesian product of its two arguments, performs a selection forcing equality on those attributes that appear in both relation schemas, and finally removes duplicate attributes.

Although the definition of natural join is complicated, the operation is easy to apply. As an illustration, consider again the example "Find the names of all customers who have a loan at the bank, and find the amount of the loan." We express this query

customer-name
Hayes
Jones
Smith

Figure 3.20 Customers with both an account and a loan at the bank.

customer-name	*loan-number*	*amount*
Adams	L-16	1300
Curry	L-93	500
Hayes	L-15	1500
Jackson	L-14	1500
Jones	L-17	1000
Smith	L-23	2000
Smith	L-11	900
Williams	L-17	1000

Figure 3.21 Result of $\Pi_{customer\text{-}name,\ loan\text{-}number,\ amount}\ (borrower \bowtie\ loan)$.

by using the natural join as follows:

$$\Pi_{customer\text{-}name,\ loan\text{-}number,\ amount}\ (borrower \bowtie\ loan)$$

Since the schemas for *borrower* and *loan* (that is, *Borrower-schema* and *Loan-schema*) have the attribute *loan-number* in common, the natural-join operation considers only pairs of tuples that have the same value on *loan-number*. It combines each such pair of tuples into a single tuple on the union of the two schemas (that is, *customer-name, branch-name, loan-number, amount*). After performing the projection, we obtain the relation in Figure 3.21.

Consider two relation schemas R and S—which are, of course, lists of attribute names. If we consider the schemas to be *sets*, rather than lists, we can denote those attribute names that appear in both R and S by $R \cap S$, and denote those attribute names that appear in R, in S, or in both by $R \cup S$. Similarly, those attribute names that appear in R but not S are denoted by $R - S$, whereas $S - R$ denotes those attribute names that appear in S but not in R. Note that the union, intersection, and difference operations here are on sets of attributes, rather than on relations.

We are now ready for a formal definition of the natural join. Consider two relations $r(R)$ and $s(S)$. The **natural join** of r and s, denoted by $r \bowtie s$, is a relation on schema $R \cup S$ formally defined as follows:

$$r \bowtie s = \Pi_{R \cup S}\ (\sigma_{r.A_1 = s.A_1 \wedge r.A_2 = s.A_2 \wedge \ldots \wedge r.A_n = s.A_n}\ r \times s)$$

where $R \cap S = \{A_1, A_2, \ldots, A_n\}$.

Because the natural join is central to much of relational-database theory and practice, we give several examples of its use.

branch-name
Brighton
Perryridge

Figure 3.22 Result of $\Pi_{branch\text{-}name}(\sigma_{customer\text{-}city\ =\ \text{“Harrison”}}\ (customer \bowtie\ account \bowtie\ depositor))$.

- Find the names of all branches with customers who have an account in the bank and who live in Harrison.

$$\Pi_{branch\text{-}name} \left(\sigma_{customer\text{-}city = \text{“Harrison”}} \left(customer \bowtie account \bowtie depositor\right)\right)$$

The result relation for this query appears in Figure 3.22.

Notice that we wrote *customer* $\bowtie$ *account* $\bowtie$ *depositor* without inserting parentheses to specify the order in which the natural-join operations on the three relations should be executed. In the preceding case, there are two possibilities:

- $(customer \bowtie account) \bowtie depositor$
- $customer \bowtie (account \bowtie depositor)$

We did not specify which expression we intended, because the two are equivalent. That is, the natural join is **associative**.

- Find all customers who have *both* a loan and an account at the bank.

$$\Pi_{customer\text{-}name} \left(borrower \bowtie depositor\right)$$

Note that in Section 3.2.3.1 we wrote an expression for this query by using set intersection. We repeat this expression here.

$$\Pi_{customer\text{-}name} \left(borrower\right) \cap \Pi_{customer\text{-}name} \left(depositor\right)$$

The result relation for this query appeared earlier in Figure 3.20. This example illustrates a general fact about the relational algebra: It is possible to write several equivalent relational-algebra expressions that are quite different from one another.

- Let $r(R)$ and $s(S)$ be relations without any attributes in common; that is, $R \cap S = \emptyset$. ($\emptyset$ denotes the empty set.) Then, $r \bowtie s = r \times s$.

The *theta join* operation is an extension to the natural-join operation that allows us to combine a selection and a Cartesian product into a single operation. Consider relations $r(R)$ and $s(S)$, and let θ be a predicate on attributes in the schema $R \cup S$. The **theta join** operation $r \bowtie_\theta s$ is defined as follows:

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

3.2.3.3 The Division Operation

The **division** operation, denoted by $\div$, is suited to queries that include the phrase “for all.” Suppose that we wish to find all customers who have an account at *all* the branches located in Brooklyn. We can obtain all branches in Brooklyn by the expression

$$r_1 = \Pi_{branch\text{-}name} \left(\sigma_{branch\text{-}city = \text{“Brooklyn”}} \left(branch\right)\right)$$

The result relation for this expression appears in Figure 3.23.

branch-name
Brighton
Downtown

Figure 3.23 Result of $\Pi_{branch\text{-}name}(\sigma_{branch\text{-}city = \text{“Brooklyn”}}\ (branch))$.

We can find all (*customer-name, branch-name*) pairs for which the customer has an account at a branch by writing

$$r_2 \ = \ \Pi_{customer\text{-}name,\ branch\text{-}name}\ (depositor \bowtie\ account)$$

Figure 3.24 shows the result relation for this expression.

Now, we need to find customers who appear in r_2 with *every* branch name in r_1. The operation that provides exactly those customers is the divide operation. We formulate the query by writing

$$\Pi_{customer\text{-}name,\ branch\text{-}name}\ (depositor \bowtie\ account)$$
$$\div\ \Pi_{branch\text{-}name}\ (\sigma_{branch\text{-}city = \text{“Brooklyn”}}\ (branch))$$

The result of this expression is a relation that has the schema (*customer-name*) and that contains the tuple (Johnson).

Formally, let $r(R)$ and $s(S)$ be relations, and let $S \subseteq R$; that is, every attribute of schema S is also in schema R. The relation $r \div s$ is a relation on schema $R - S$ (that is, on the schema containing all attributes of schema R that are not in schema S). A tuple *t* is in $r \div s$ if and only if both of two conditions hold:

1. t is in $\Pi_{R-S}(r)$
2. For every tuple t_s in s, there is a tuple t_r in r satisfying both of the following:
   a. $t_r[S] = t_s[S]$
   b. $t_r[R - S] = t$

It may surprise you to discover that, given a division operation and the schemas of the relations, we can, in fact, define the division operation in terms of the fundamental operations. Let $r(R)$ and $s(S)$ be given, with $S \subseteq R$:

$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

customer-name	*branch-name*
Hayes	Perryridge
Johnson	Downtown
Johnson	Brighton
Jones	Brighton
Lindsay	Redwood
Smith	Mianus
Turner	Round Hill

Figure 3.24 Result of $\Pi_{customer\text{-}name,\ branch\text{-}name}\ (depositor \bowtie\ account)$.